Running Jobs

On the Job Scheduler Overview page we described how to run both batch and interactive jobs. This page describes how to request resources for your job.

Cores for Multicore and Shared Memory Jobs

For any kind of multithreading, multiprocessing, shared memory, OpenMP, or similar jobs you can use the `-c` or `--cpus-per-task` flag to request additional cores.

An example batch script that requests 4 cores on the mit_normal partition would be:

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal
#SBATCH -c 4

# Set up environment
module load miniforge

# Run your application
python myscript.py
```

MPI and Distributed Jobs

There are a few flags to be aware of for running distributed and MPI jobs. These types of jobs can run across multiple nodes, so there are a few flags that can control how these tasks are spread out.

- `-n` or `--ntasks`: This flag is used to specify the total number of distributed or MPI processes or ranks that your application uses. "Task" is roughly the term that Slurm uses for a process. By default you get `ntasks` number of cores for your job, one per task.
- `-N` or `--nodes`: This is the number of nodes your tasks are spread across. If you don't specify `ntasks` or `ntasks-per-node` you will get one per node. If you specify `ntasks` they may be split roughly evenly across `nodes`, but not necessarily. It is generally more efficient for processes on the same node to communicate, so in practice you often want to avoid spreading your tasks across more nodes than necessary.

- `--ntasks-per-node`: Use with `--nodes`, this specifies how many tasks should be assigned to each node.

At minimum you'll need to use the `-n` or `--ntasks` flag. This job here runs an MPI job with 4 processes:

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal
#SBATCH -n 4

# Set up environment
module load openmpi

# Run your application
mpirun my_program
```

Note that `mpirun` does not necessarily need to be told how many how many processes to start, it can get this information from the scheduler.

When your MPI job gets too big to fit on a single node, or you want to have more control over how your processes are spread out, use the `--nodes` and `--ntasks-per-node`. This script runs an MPI job that requests 2 nodes and 64 tasks per node, for a total of 128 MPI processes.

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=64

# Set up environment
module load openmpi

# Run your application
mpirun my_program
```

The following will still allocate 128 MPI processes across 2 nodes, but they may not necessarily be evenly split between the two nodes.

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal
#SBATCH --nodes=2
#SBATCH --ntasks=128
```

```
# Set up environment
module load openmpi

# Run your application
mpirun my_program
```

Memory

Each job is allocated some default amount of memory, the amount depending on the node. If you find that you are running out of memory you can request more with the `--mem` or `--mem-per-core` flags.

Here is an example of a job multicore job that requests 4 cores and 16GB of RAM:

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal
#SBATCH -c 4
#SBATCH --mem=16G

# Set up environment
module load miniforge

# Run your application
python myscript.py
```

How do you know how much memory your job needs? You can find out how much memory a job used after the job is completed. First, run your job with your best estimate for how much memory you need (you can overestimate), and run the job long enough to get an idea of the memory requirement. If the job fails run the job again requesting more memory. Then you can use the `sacct` slurm command to get the memory used:

```
sacct -j JOBID -o JobID,JobName,State,ReqMem,MaxRSS --units=G
```

where `JOBID` is your job ID. `State` shows the job status, keep in mind that the amount of memory reported by `sacct` is only accurate for jobs that are no longer running, and `ReqMem` is the number of memory that was allocated to the job. `MaxRSS` is the maximum resident memory (maximum memory footprint) used by each job.

If the `MaxRSS` value is larger than what you have requested, or you run out of memory, you will have to request additional memory for your job.

GPUs

GPUs are available through the `mit_normal_gpu` partition. You can also run preemptable GPU jobs on the `mit_preemptable` partition.

To request a GPU use the `-G` or `--gpus` flag and specify the number of GPUs that you need for the job (ex: `-G 1`). If you need a specific type of GPU specify that type with the pattern `-G [GPU_TYPE]:[#GPUS]`.

For example, to request 1 L40S GPU, use the flag `-G l40s:1`. To request 2 H200 GPUs use the flag `-G h200:2`. If you don't specify a GPU type on `mit_normal_gpu` or `mit_preemptable` you will be allocated an L40S GPU. L40S GPUs are good for most machine learning workloads. H200s should be reserved for jobs that require more memory than an L40S or require high precision (float 64).

An example batch script that requests 4 cores and 1 GPU on the `mit_normal_gpu` partition to run a Python script would be:

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal_gpu
#SBATCH -c 4
#SBATCH -G 1

# Set up environment
module load miniforge

# Run your application
python myscript.py
```

Requesting too many CPUs may use up your allocation

To make sure there are enough CPUs for each GPU to accept jobs, we reserve a certain number of CPUs for each GPU. If you request too many CPUs (more than the number of CPUs divided by number of GPUs), then your job may not run or may be allocated an extra GPU, which will count against your allocation. See the table below for the number of CPUs available per GPU on `mit_normal_gpu`.

If you are running a multi-node (distributed) GPU job use the `--gpus-per-node` flag and specify the number of GPUs you need per node rather than the total number of GPUs.

You can see how many GPUs of which type are on each node in a partition using the `sinfo` command. For example, to check `mit_normal_gpu` run the command:

```
sinfo -p mit_normal_gpu -O Partition,Nodes,CPUs,Memory,Gres
```

Below is a summary of the GPU nodes in `mit_normal_gpu`. See the Available Resources page for more details.

GPU Type	Number of GPUs on Node	GPU Memory	Number of CPUs on Node	Available CPUs per GPU
L40S	4	44GB	64	16
H200	8	140GB	120	15

November 19, 2025